

CoE Cheat Sheet - Liquibase and SQLCL database scripting and installation



Useful Links

- <https://www.liquibase.com/> - Liquibase product home page
- [SQLCL and Liquibase from Oracle.com](#)
- [CoE Best Practice Database Design](#)

- [1. Overview](#)
- [2. General Scripting Guidance](#)
- [3. Object and Change Script examples](#)
 - [3.1. Soft Object](#)
 - [3.2. Dropping Soft Objects](#)
 - [3.3. Hard Objects](#)
- [4. More Complex Scenarios - Dependent Objects](#)
 - [4.1. Scenario - Dropping a table](#)
 - [4.2. Scenario - Amending the columns in a Primary Key](#)
- [5. Repo Structure](#)

1. Overview

This document is a cheat sheet for the most common liquibase concepts for engineers delivering scripts for database content. The repo structure and more importantly the controllers are not covered in this document.

2. General Scripting Guidance

Statement	Narrative
Standards and Best Practice	You MUST consider and apply best practice and standards when defining your repo, designing your objects and when scripting the installation!
Avoid DROP and TRUNCATE	Whilst there are legitimate scenarios where a DROP or TRUNCATE are required, these are exceptional. Always discuss and agree with peers before using either of these or any other destructive commands
Repo filenames should reflect object names	The repo structure will segregate object types into schemas and object types. The file names should reflect the object names e.g. tablename, constraintname, indexname but not the action type i.e. not create_tablename, this is what the changeset defines. Group changesets into a script file for a single object e.g. table and column comments can be applied to a single file named with the tablename. Finding things in the repo and the change logs should be simple.
All executable scripts need a liquibase header.	The header is used by liquibase to identify an executable file. If this is not added the script will not run. <pre>--liquibase formatted sql</pre>
All installation files should use the "sql" file extension	SQLCL will not execute files without the sql extension e.g. pkb, pks. This is another reason (in addition to executing spec before body) to host the package spec and package body files in different sub directories
Changesets should be unique and descriptive	Changesets consist of an author (use your guid) and an ID. The ID should be unique across your entire changelog. The ID should describe the action being taken to make it easy to interrogate the change log and to easily understand the changelog execution. The meaningful description also helps maintain uniqueness. Consider the following table creation script for table named "HRS_JOB_ROLE". • "chrthoms:create_table" - BAD - There will be lots of tables created! • "chrthoms:hrs_job_role" - BAD - You may well have other objects with the same name e.g. synonym • "chrthoms:create_table_hrs_job_role" - GOOD - this is more descriptive and more likely to be unique. Adding the schema/owner may be even more beneficial!
Use one changeset per commit	Each DDL statement implicitly commits. Imagine the scenario where you have a single changeset that creates a table then alters it twice. First execution sees the table being created but the first alter fails. You correct the script and re-install. This time the create table fails as it exists. You have created a scenario that you cannot escape from. Using a single changeset for each statement / each commit avoids this risk. By using different changesets you can also use the liquibase logs to confirm that all of the changes were executed.
Always consider existing data when altering an object	As your data model transforms existing data must be considered. You should never damage existing data during an install. You should also consider performance i.e. adding an index or constraint on a multi-million row table may need a slightly different approach. Adding a not null column to a table with data or altering a constraint may fail based on existing data!

Use an appropriate run on change value	If the object you are installing is soft object (supports a create or replace) e.g. view or package body you can and should use a "runOnChange:true". By exception you may never want to re-execute a create or replace on change. If adding or altering a hard object (supported only create or later) e.g. table or constraint you should use "runOnChange:false" or exclude the runOnChange entirely. Never use "runOnChange:true" for a hard object (A hard object is one that is created or amended e.g. table versus a soft object which is created or replaced e.g. view). An additional replacement for "runOnChange" is "runAlways". This defaults to false and is only set to true by exception. This is most typically implemented on pre or post install steps e.g. recompile all schemas, where we always want the script to be executed.
Pre-conditions and hard objects	A changeset without a pre-condition and not set to run on change will execute once. This approach can be used for hard objects. We strongly recommend using pre-conditions. Pre-conditions allow you to define the scenario when you want your change to be applied. If we clone an environment, lose the liquibase logs or if another script changes our object a pre-condition can ensure that we always result in the expected state. The purpose of the installation is to get from any initial state to a known end state and pre-conditions support this. Pre-conditions are applied after your changeset. These typically interrogate one or more data dictionary views looking for the no-existence or existence of object or a setting e.g. column exists and is a varchar of 25 then increase to varchar 35. Complex combinations of settings can also be applied e.g. if the table exists, column exists, column is a varchar 25 and not null then execute a script to make the column 30 characters and nullable. It is recommended to make pre-condition logic to return a binary result using the logic to hide any complexities. <pre>--preconditions onFail:MARK_RAN onError:HALT --precondition-sql-check expectedResult:0 select count(1) from all_tables where owner = 'MYONWER' and table_name = 'MYTABLE';</pre> Avoid using anonymous blocks, these will not be logged correctly and are more difficult to test. Liquibase change logs will provide an exectype. - EXECUTED is applied when the script is ran and applies to the first execution only, this may would reflect a pre-condition was met and the script was executed - RERAN applies to a script which has been executed before e.g. a view that was installed then subsequently changed. - MARK_RAN is where a log for the changeset did not exist before, the pre-condition was not met so the sql was not executed.
ALWAYS install the script files. NEVER assume the changes will work!	Your script may look well formed. It may run when you execute it in SQL. Until you execute with liquibase you have not proven the change! The liquibase marup may not work as you expected, the controller may execute the script in a sequence you did not intend, you may hit a dependency. Ideally you will install from a known start point i.e. last prod release and also from a clean install. Ideally you will have data loaded to prove any transformation or constraint issues you may face. Always try to script in files and install. Try to avoid ever running DDL statements manually. Before installing the database should be in an installed state i.e. no manual changes. Try changes out in another schema. If that is not possible then revert any/all manual changes before installing. You can use the data dictionary and the liquibase logs to confirm that the expected changes were executed and applied as expected.
Apply a Rollback that reinstates an earlier state	If you create an object drop it, if you alter an object alter it back. Rollbacks are problematic as data may have changed, objects may have dependencies. Whilst we don't intend on rolling back, the rollback can help engineers when diagnosing/fixing a failed install in dev.

3. Object and Change Script examples

3.1. Soft Object

These are objects that can be created or replaced or in some cases created even if they exist without failing.



In all of these examples you will typically use a "runOnChange=true". Altering the script will be sufficient to cause it to re-execute on the next installation.

Object Type	Example Script / Snippet	Notes
-------------	--------------------------	-------

Package Spec	<pre>--liquibase formatted sql --changeset myguid: package_name_spec_create stripComments:false endDelimiter: / runOnChange:true create or replace package owner. package_name as --- --- end package_name / --rollback drop package owner. package_name;</pre>	The end delimiter is /. After the end of a package we always take a new line then terminate with a forward slash. We add rollback but you can probably appreciate why we don't rollback as part of the installation, rolling back does not re-instate the last package state.
Package Body	<pre>--liquibase formatted sql --changeset myguid: package_name_body_create stripComments:false endDelimiter: / runOnChange:true create or replace package body owner.package_name as --- --- end package_name / --rollback drop package body owner.package_name;</pre>	The end delimiter is /. After the end of a package we always take a new line then terminate with a forward slash.
Synonym	<pre>--liquibase formatted sql --changeset myguid: synonymowner_object_name_synonym_ create stripComments:false create or replace synonym synonymowner.object_name for objectowner.object_name; --rollback drop synonym synonymowner.object_name;</pre>	There is no end delimiter. ";" is the default and does not need to be defined. For synonyms we want to reference the owning schema. This is due to synonyms being used to expose objects across schemas.
View	<pre>--liquibase formatted sql --changeset myguid: view_viewname_create stripComments:false runOnChange: true create or replace force view owner.view_name as --query content ; --rollback drop view owner. view_name;</pre>	Always use the FORCE keyword on a view. If a view references an invalid object or an object you don't yet have privileges on during creation it will fail. The FORCE keyword forces the view to create allowing you to compile and validate at a later stage in the installation process.

Grant	<pre>--liquibase formatted sql --changeset myguid: privilege1_privilege2_on_objectname_to_grantee stripComments: false runOnChange:true grant privilege1, privilege2 on owner.object_name to grantee; --rollback revoke privilege1, privilege2 on owner.object_name from grantee;</pre>	Do not be tempted to add multiple grants into a single changeset. A change to any one grant will cause all of the grants to be re-applied. You will also lose the ability to track the grants being applied in the change log. It is acceptable to grant multiple privileges in one DDL statement
Comments	<pre>--liquibase formatted sql --changeset myguid: tablename_comments stripComments: false runOnChange:true endDelimiter;; comment on table owner.tablename is 'my table comment'; comment on column owner. tablename.columnname1 is 'my column comment'; comment on column owner. tablename.columnname2 is 'my column comment'; ----</pre>	Comments do not alter the logic or behavior. This is the exception where a single changeset can be used for multiple DDL statements. We don't need a rollback on comments.

3.2. Dropping Soft Objects

Object Type	Example Script / Snippet	Notes
Package	<pre>--liquibase formatted sql --changeset myguid:package_packagename_drop --preconditions onFail:MARK_RAN onError:HALT --precondition-sql-check expectedResult:1 select count(1) from all_objects where owner='OWNER' and object_type = 'PACKAGE' and object_name = 'PACKAGENAME'; drop package owner.packagename;</pre>	When dropping a package drop you should drop the package in the spec file. Do NOT create then drop to the package, remove the changeset for the creation. Adding a pre-condition ensures that the package is removed when it exists and only if it exists. Delete the package body file from the repo. Always consider any dependent objects or scripts e.g. grants or synonyms. Grants should be deleted/removed from the repo, dropping an object will remove the grant. Synonyms should be dropped.

View	<pre>--liquibase formatted sql --changeset myguid:view_viewname_drop --preconditions onFail:MARK_RAN onError:HALT --precondition-sql-check expectedResult:1 select count(1) from all_objects where owner='OWNER' and object_type = 'VIEW' and object_name = 'VIEWNAME'; drop view owner.viewname;</pre>	When dropping a view use the same repo file but remove the original changeset. Do NOT create then drop. Adding a pre-condition ensures that the view is only removed when it exists. Always consider dependencies e.g. grants or synonyms.
Synonym	<pre>--liquibase formatted sql --changeset myguid:view_synonymname_drop --preconditions onFail:MARK_RAN onError:HALT --precondition-sql-check expectedResult:1 select count(1) from all_objects where owner='OWNER' and object_type = 'SYNONYM' and object_name = 'SYNONYMNAME'; drop synonym owner.synonymname;</pre>	When dropping a synonym use the same repo file but remove the original changeset. Do NOT create then drop. Adding a pre-condition ensures that the synonym is only removed when it exists. Always consider dependencies e.g. grants.
Grant		Grants are not dropped. We revoke grants. This is described in the next section on hard objects.
Comments		We never need to drop comments. We either adjust the comment content, remove a column if it dropped, add a comment for a new comment or if we drop the table we simply delete this file from the repo (comments drop with the table).

3.3. Hard Objects

These are object that can only be created or altered, where each DDL statement can only execute once.



If you are unsure of how to script and liquify changes ask an expert. You have lots of experienced colleagues who will be happy to share their experience. Discussing your planned approach or asking for guidance is encouraged.

Object Type	Change Type	Example Script / Snippet	Notes
-------------	-------------	--------------------------	-------

Table	Create	<pre>--liquibase formatted sql --changeset myguid: table_tablename_create stripComments: false --preconditions onFail:MARK_RAN onError: HALT --precondition-sql-check expectedResult: 0 select count(1) from all_tables where owner = 'OWNER' and table_name = 'TABLENAME'; CREATE TABLE owner.tablename(pk_column VARCHAR2(50 CHAR) NOT NULL, column1 VARCHAR2(256 CHAR) NOT NULL, column2 VARCHAR2(2000 CHAR) NOT NULL, row_version NUMBER DEFAULT 0 NOT NULL, created_by VARCHAR2(255 CHAR) NOT NULL, created_on TIMESTAMP WITH TIME ZONE DEFAULT current_timestamp NOT NULL, updated_by VARCHAR2(255 CHAR) NOT NULL, updated_on TIMESTAMP WITH TIME ZONE DEFAULT current_timestamp NOT NULL) LOGGING; --rollback drop table owner.tablename;</pre>	Only define the table columns with datatypes, precision, defaults and nullability. Do NOT add constraints or other table related objects.
Constraint	Create Primary Key	<pre>--liquibase formatted sql --changeset myguid: constraint_tablealias_pk_create stripComments:false --preconditions onFail:MARK_RAN onError: HALT --precondition-sql-check expectedResult: 0 select count(1) from all_constraints where owner = 'OWNER' and constraint_name = 'TABLEALIAS_PK'; alter table owner.tablename add constraint tablealias_pk primary key (pk_column); --rollback alter table owner.tablename drop constraint tablealias_pk;</pre>	Never add constraints as part of table creation. Constraints should be in their own controller folder. PK's should be applied after indexes have been created. The PK constraint and PK index are different object types and should be created in different files in different repo locations.

Constraint	Create Foreign Key	<pre>--liquibase formatted sql --changeset myguid: constraint_parenttablealias_childtablealiasb_fk1_create stripComments:false --preconditions onFail:MARK_RAN onError: HALT --precondition-sql-check expectedResult: 0 select count(1) from all_constraints where owner = 'OWNER' and constraint_name = 'PARENTTABLEALIAS_CHILDTABLEALIAS_FK1' alter table owner.parenttablename add constraint parenttablealias_childtablealiasb_fk1 foreign key (column1,column2) references owner.childtablename(column1, column2); --rollback alter table owner. parenttablename drop constraint parenttablealias_childtablealiasb_fk1;</pre>	Never add constraints as part of table creation. Constraints should be in their own controller folder. FK depend on two tables and both need to exist before creation. The FK constraint and FK index are different object types and should be created in different files in different repo locations.
Constraint	Create Unique	<pre>--liquibase formatted sql --changeset myguid: constraint_tablealias_uk1_create stripComments:false --preconditions onFail:MARK_RAN onError: HALT --precondition-sql-check expectedResult: 0 select count(1) from all_constraints where owner = 'OWNER' and constraint_name = 'TABLEALIAS_UK1' alter table owner.tablename add constraint tablealias_uk1 unique (column1,column2); --rollback alter table owner.tablename drop constraint tablealias_uk1;</pre>	The UK constraint and UK index are different object types and should be created in different files in different repo locations. The UK constraint and UK index are different object types and should be created in different files in different repo locations.

Constraint	Create Check	<pre>--liquibase formatted sql --changeset myguid: constraint_tablealias_ck4_create stripComments:false --preconditions onFail:MARK_RAN onError: HALT --precondition-sql-check expectedResult: 0 select count(1) from all_constraints where owner = 'OWNER' and constraint_name = 'TABLEALIAS_CK1' alter table owner.tablename add constraint tablealias_ck4 check (column3_YN in ('Y','N')); --rollback alter table owner.tablename drop constraint tablealias_ck4;</pre>	
Index	Create Unique	<pre>--liquibase formatted sql --changeset myguid: index_tablealias_uk1_create stripComments:false --preconditions onFail:MARK_RAN onError: HALT --precondition-sql-check expectedResult: 0 select count(1) from all_indexes where owner = 'OWNER' and index_name = 'TABLEALIAS_UK1' CREATE UNIQUE INDEX owner. tablealias_uk1 ON owner.tablename (column1 asc) online; --rollback drop index owner. tablealias_uk1;</pre>	The UNIQUE keyword is critical to enforcing the uniqueness. Unique constraint and primary key indexes are created in the same way.

Index	Create non-Unique	<pre>--liquibase formatted sql --changeset myguid: index_tablealias_fk1_create stripComments:false --preconditions onFail:MARK_RAN onError: HALT --precondition-sql-check expectedResult: 0 select count(1) from all_indexes where owner = 'OWNER' and index_name = 'TABLEALIAS_FK1' CREATE INDEX owner.tablealias_fk1 ON owner.tablename (column1 asc, column 2 desc) online; --rollback drop index owner. tablealias_fk1;</pre>	
Column	Change Precision	<pre>--liquibase formatted sql --changeset myguid: column_tablealias_columnname_precision_alter stripComments:false --preconditions onFail:MARK_RAN onError: HALT --precondition-sql-check expectedResult: 0 select count(1) from all_tab_columns where owner = 'OWNER' and table_name = 'TABLENAME' and column_name = 'COLUMN1' and data_length != 30; alter table owner.tablename modify column1 varchar2(30 char); --rollback alter table owner.tablename modify column1 varchar2(10 char);</pre>	Change precision from 10 to 30. A column alteration should be added as a changeset to the existing table script file, it can be considered part of the same object. The pre-condition that verifies the current length is critical to stop the changeset re-running. This pre-condition assumes that the column exists, we want this to fail if the column does not exist. There may be scenarios where we want the pre-condition to be more explicit interrogating multiple data dictionary views and collating a final result to check against. If the column is part of a foreign key you may not be able to change without altering all other referential columns. This is a complex cross object change and needs consideration, not covered in this cheat sheet. Speak to the experts to agree the correct method.
Column	Add	<pre>--liquibase formatted sql --changeset myguid: column_tablealias_columnname_add stripComments:false --preconditions onFail:MARK_RAN onError: HALT --precondition-sql-check expectedResult: 0 select count(1) from all_tab_columns where owner = 'OWNER' and table_name = 'TABLENAME' and column_name = 'COLUMN2'; alter table owner.tablename add column2 varchar2(30 char); --rollback alter table owner.tablename drop column2;</pre>	A column alteration should be added as a changeset to the existing table script file, it can be considered part of the same object. A pre-condition should be applied to only add the column if it doesn't already exist.

Column	Not Null	<pre>--liquibase formatted sql --changeset myguid: column_tablealias_columnname_null_alter stripComments:false --preconditions onFail:MARK_RAN onError: HALT --precondition-sql-check expectedResult: 0 select count(1) from all_tab_columns where owner = 'OWNER' and table_name = 'TABLENAME' and column_name = 'COLUMN1' and nullable = 'N'; alter table owner.tablename modify column1 varchar2(30 char) null; --rollback alter table owner.tablename modify column1 varchar2(30 char) not null;</pre>	A column alteration should be added as a changeset to the existing table script file, it can be considered part of the same object. A pre-condition should be applied to only alter the column if it isn't already nullable
Column	Drop	<pre>--liquibase formatted sql --changeset myguid: column_tablealias_columnname_drop_alter stripComments:false --preconditions onFail:MARK_RAN onError: HALT --precondition-sql-check expectedResult: 0 select count(1) from all_tab_columns where owner = 'OWNER' and table_name = 'TABLENAME' and column_name = 'COLUMN2'; alter table owner.tablename drop column column2; --rollback alter table owner.tablename add column2 varchar2(30 char);</pre>	A column alteration should be added as a changeset to the existing table script file, it can be considered part of the same object. A pre-condition should be applied to only add the column if exists. Dropping columns could have an impact on other objects including table related like constraints and indexes but also views or packages. Always consider the full impact and alter all other related scripts. It is acceptable to drop the column temporarily invalidating these dependent objects, a well formed install will revalidate all objects.
Column	Rename	<pre>--liquibase formatted sql --changeset myguid: column_tablealias_newcolumnname_rename_alter stripComments:false --preconditions onFail:MARK_RAN onError: HALT --precondition-sql-check expectedResult: 0 select count(1) from all_tab_columns where owner = 'OWNER' and table_name = 'TABLENAME' and column_name = 'COLUMN2'; alter table owner.tablename rename column column2 to column3; --rollback alter table owner.tablename rename column column3 to column2;</pre>	A column alteration should be added as a changeset to the existing table script file, it can be considered part of the same object. A pre-condition should be applied to only add the column if exists. Consider all dependencies i.e. indexes, constraints, packaged code, views, when renaming a column. Installing will help you identify any scripts you missed, some may fail, the install object status check will highlight others.

Grant	Revoke	<pre>--liquibase formatted sql --changeset myguid: grant_privilegename_on_viewname_from_grantee_revoke runOnChange:true --preconditions onFail:MARK_RAN onError:HALT --precondition-sql-check expectedResult:1 select count(1) from dba_tab_privs where owner = 'OWNER' and table_name = 'VIEWNAME' and grantee = 'GRANTEESchema' and privilege = 'SELECT'; revoke select on owner.viewname from granteeschema; --rollback grant select on owner.viewname to granteeschema;</pre>	Revoke select from a view. A grant can only be revoked if applied. You will get an error if you remove a grant that is not currently applied. The pre-condition is critical to avoid that failure.  The dba_tab_privs view table_name column can contain an object type, tables, views and even packages
-------	--------	--	---

4. More Complex Scenarios - Dependent Objects

Where you have cross object dependencies you must take a considered approach to ensure that your scripts always produce the expected outcome whilst removing risk of failures. Always work with a peer to define all the scenarios you may encounter and need to handle, considering the dependent objects and any changes they may need. Also consider how this will work in a clean no object install versus a previous version being installed. Ask yourself how this will work when data is present. A peer will help you identify scenarios you may not have considered.

When altering the definition of an index or constraint it is generally better to remove the original and replace it with a new version altering the name (increment the number e.g. FK1 becomes FK2 or CK3 becomes CK4) rather than dropping and recreating with the same name. The differentiation in names makes the pre-conditions simpler and more precise and supports schema content validation i.e. we know that version 2 has CK1 and version 3 has CK2 and not CK1. Differentiating between CK1 variant 1 and 2 is more complex and brittle. Schema verification is not in place yet but will be soon.

Below are two examples where complexity means taking a less obvious approach.

4.1. Scenario - Dropping a table

Assuming the table has constraints, indexes, comments, grants and a synonym and a table api package and an api view.

First of all should we drop the table? Might we need to retain it for support, do we just remove the interfaces to it?

- We would alter the table script to
 - deleting the table creation changeset
 - delete any alter table changesets
 - add a changeset to drop table pre-conditioned to only execute if the table existed
- When a table is dropped the constraints, indexes, grants and comments are automatically dropped. We would need to delete all of these files from the repo to ensure they don't execute at a later date if the existing pre-condition states create if it doesn't exist!
- The spec for the table api package should be changed to a drop package pre-conditioned on the existence of the package, removing the create package changeset. The package body script file can be deleted.
- The api view script should be changed to drop the view pre-conditioned on its existence removing the original changeset to create the view.
- Any synonyms on the table, Api view or table Api package should also be changed to drop if they exist removing the create changesets.

We may invalidate other objects that rely on the dropped table. Impact assessment and code changes should be applied to remedy this. After installing the changes the objects should all be recompiled and validity confirmed. A well formed repo will always contain a recompile and confirm status step.

Note that we do not need to drop all of the objects, some can be based on script file deletion. We would never want to retain the original changesets, only the latest changset that you want to install.

4.2. Scenario - Amending the columns in a Primary Key

The Primary key consists of a constraint and an index. We recommend using a different PK constraint and index name e.g. suffixed with PK1, PK2 depending on the number of iterations that have taken place already.

We need to consider the uniqueness of the data before proceeding, not only in the dev environment but every other environment especially customer production environments.

Assuming the liquibase controllers run index scripts before constraint scripts we would need to

- Alter the current create PK index script to
 - changeset to disable the current PK constraint where it exists
 - changeset to drop the index where it exists
- Add a new script to create the new PK index
- Alter the existing PK constraint script to
 - remove the original create constraint changeset
 - changeset to drop the constraint where it exists
- Add a new script and changeset to create the new PK constraint

5. Repo Structure

The Repo Structure can help us manage the sequence of installation and should support multiple schemas and object types.

Files in the repo should use the object_name.sql notation.

Files will generally contain a single object i.e. tables and indexes will not be in the same file. Grants and seed data can be exceptions where all the grants for an object can be applied in a single file. There are exceptions when a specific sequence of changeset execution needs to be managed. These exceptions should be discussed and agreed with peers to make sure we need the exception, these will typically be more complex transformational changes.

The following base structure allows for control of the installation across multiple schemas

1. product
 - a. Pre_install - This contains logic that is needed before the install e.g. disabling jobs or setting up logs
 - b. db - contains database code, apex may also be relevant in some repos
 - i. _dba - DBA scripts like sys privileges or schema creation
 - ii. schema-1 - all the content for a single schema
 - iii. schema-n - repeat for all schemas
 - c. post_install - general changes like compiling invalid object
 - d. Install_validation -
 - i. pre_validation - creating objects needed by the validator
 - ii. validation - individual validations that need to be executed e.g. are all object valid, is the seed data correct, do we have the expected objects created
 - iii. post_validation - Confirm the install e.g. log the version or call OCI to expose the version
2. non_product - this section contains any logic needed by developers or for testing that should never be installed in production.

The repo should have a productController.xml file to orchestrate the entire installation and a non_productController .xml for the non product content. Ideally the non_product content will not exist in the production artefact, accidentally allowing it to be deployed. This will need to be managed in the build stages.

Each sub section will need its own controllers to execute the content in that section e.g. schemaController.xml will execute the objects in the correct sequence.

The schema structures should support all objects. The following example covers most object types but can be extended to support others. The sequence indicates the install sequence to be applied in the schema controller.

1. pre_install
2. privilege
3. sequence
4. table
5. index
6. constraint pk
7. constraint uk
8. constraint other
9. constraint fk
10. temp_table
11. type_spec
12. pkg_spec
13. view
14. mv
15. type_body
16. pkg_body
17. trigger
18. function - These should be avoided prefer packaged content
19. procedure - These should be avoided prefer packaged content
20. schedule
21. job
22. synonym
23. grant
24. role
25. seed_data
26. rest_module
27. comment
28. post_install